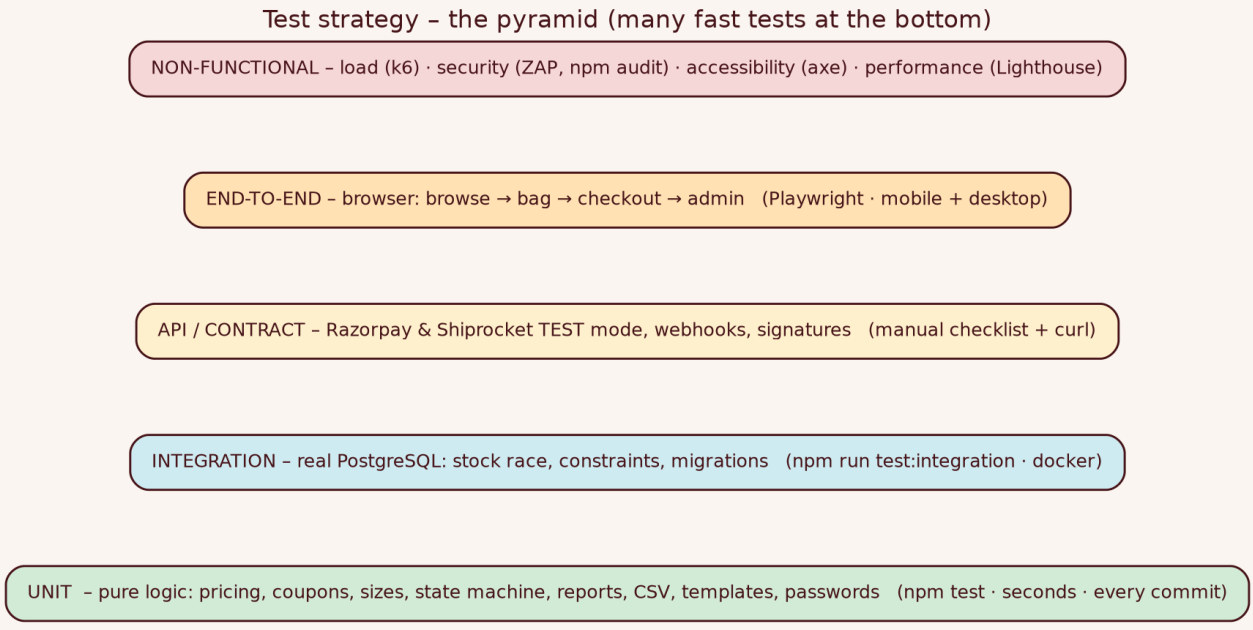
LUXE & THREAD · DOCUMENTATION

INTERNAL – for developers

Internal 11 · Test Strategy & Test Plan

Audience	INTERNAL – developers, QA
Goal	Every kind of useful test, how to run it, what "pass" means



1 · Status today

Level	Tool	Count	Status
Static: types	`tsc --noEmit` (strict)	whole project	✅ passes
Static: lint	ESLint 9	whole project	✅ passes
Unit	Node test runner	**23**	✅ pass (pricing, coupons, sale windows, stock rules, sizes, state machine, cancellation policy, CSV, reports in IST, templates, passwords, sessions, signatures, rate-limit, logger, dates, media)
Integration (real PostgreSQL)	node:test + `pg`	5	🟡 written; run with Docker (§3)
End-to-end	Playwright	7 scenarios × 2 devices	🟡 written; run against staging (§4)
Load	k6	1 script	🟡 run on staging (§5)
Security / accessibility / performance	ZAP, axe, Lighthouse	procedures	🟡 run before go-live (§6–8)

Level	Tool	Count	Status
Provider tests (Razorpay, Shiprocket, e-mail)	manual + curl	checklist	🟡 run in TEST mode (§9)

2 · Unit tests – `npm test`

Compiles `src/lib` with TypeScript and runs `node --test`. They need **no database and no internet**. Add a test whenever you change a rule. Example scenarios covered: *sale price active only inside dates · cannot order more than stock, even split across two lines · coupon discount shares add up exactly · RTO order reverses a sale in profit report · 12:30 am IST order lands on the next day · promoting yourself to owner by editing the cookie fails*.

3 · Integration tests – real database

```
docker compose up -d db
docker compose exec db createdb -U luxe luxe_thread_test
DATABASE_URL=postgres://luxe:luxe_dev_password@localhost:5432/luxe_thread_test npm run
test:integration
```

⚠️ The test **drops the schema** of that database – never point it at real data (the file refuses URLs containing "prod"). Covers: migration + seed load · negative stock rejected by the database · **10 buyers racing for 1 piece → exactly one wins** · unique SKU / e-mail rules · invalid order status rejected.

4 · End-to-end (browser) tests

```
npm run build && npm start                  # terminal 1 (staging DB seeded with demo data)
npx playwright install --with-deps chromium  # once
npm run test:e2e                            # terminal 2 → report in playwright-report/
```

Scenarios: home & navigation · browse → pick size → add → bag shows server price · out-of-stock size disabled · coupon valid/invalid · size guide · checkout validation messages · admin redirect when logged out. Runs on desktop Chrome and a Pixel 7 phone profile.

5 · Load test

`k6 run -e BASE=https://staging.your-domain.in tests/load/shop.k6.js` — ramps to 50 virtual users for 3 minutes over home, shop, filters, sale, size guide, health and the cart-price API. **Pass:** p95 < 800 ms, errors < 1 %. Watch `pm2 monit`, `select count(*) from pg_stat_activity`. Never run against production in business hours.

6 · Security tests

Test	How	Pass
Dependency vulnerabilities	`npm audit --omit=dev`	no high/critical

Test	How	Pass
Web scan	`docker run -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t https://staging…`	no high alerts
Auth	try `/admin/orders`, `/admin/download/orders-csv` signed-out	redirect / 401
Role	staff user opens `/admin/reports`, `/admin/coupons`	redirected to dashboard with notice
Price tampering	change price in `localStorage`, check out	charged real price
Quantity/negative	POST `/api/cart/price` with quantity −1 or 999	422
SQL injection	search `?q='; drop table products;--`	normal "no results"
XSS	product name `<script>alert(1)</script>`	shown as text
CSV injection	customer name `=HYPERLINK(...)`, download CSV	cell prefixed with `'`
Upload	rename `.php` to `.jpg` and upload	rejected (magic bytes)
Webhook forgery	call webhooks without/with wrong signature/token	400/401
Rate limits	20 rapid logins	lockout message
Order link privacy	open `/order/LT…` without `?t=`	404

7 · Accessibility (WCAG 2.1 AA)

Run Lighthouse *Accessibility* ≥ 90 on home, shop, product, bag, checkout. Manual: tab through checkout with keyboard only; screen-reader labels on size buttons (`role=radio`), form errors announced (`role=alert`); colour contrast of maroon on sandalwood ✓. Optional: add `@axe-core/playwright` to the e2e specs.

8 · Performance

Lighthouse mobile: Performance ≥ 80, LCP < 2.5 s on 4G. Ensure photos ≤ 300 KB; Nginx gzip on. Check `EXPLAIN ANALYZE` of the shop query when the catalogue > 5 000 products.

9 · Provider test checklist (TEST mode)

#	Scenario	Expected
1	Pay with test card success	order PAID; e-mail sent; event `PAYMENT_CAPTURED`
2	`failure@razorpay`	message in form; bag kept
3	Fail then success	PAID
4	Close Razorpay window	"Payment was cancelled"; bag kept; order expires after 30 min, stock back
5	Block `/api/checkout/verify` after paying	order page "Confirming…" → PAID via webhook
6	Redeliver webhook	200, no duplicate events
7	Wrong webhook secret	400 INVALID_SIGNATURE
8	Cancel a PAID order	CANCELLED, stock restored, refund event, refund e-mail

#	Scenario	Expected
9	Pay after expiry (open window, wait 31 min, pay)	LATE_PAYMENT + auto refund
10	COD order (COD_ENABLED)	COD_CONFIRMED → shipment
11	Last piece race: two browsers, one size left	one succeeds, other "just sold out"
12	Wrong pickup nickname	SHIPMENT_FAILED shown; fix; retry works
13	Courier webhook IN TRANSIT → DELIVERED	statuses + e-mails; `delivered_at` set
14	Delivered order → request return → approve → received	RETURNED, restock, refund
15	SMTP wrong password	message `failed`, retried; after fix **Retry failed** → sent

10 · Regression pack before each release

`typecheck` · `lint` · `npm test` · `build` · integration · e2e on staging · provider checklist items 1, 5, 8, 11, 13 · health check §A.

11 · Test data

`npm run db:seed` (demo catalogue; coupon `WELCOME10`). Reset: drop/recreate the test database and re-run migrate + seed. Never use production customer data in tests.

12 · Defect handling

Record: steps, expected, actual, screenshot, request id, severity (S1 money/data loss · S2 feature broken · S3 cosmetic). S1/S2 block release.